



Quarkslab

SightHouse Workshop

Reverse Engineering with Function Similarity

Fenrisfulsur & MadSquirrels - 01/07/2026



Pass
the SALT

About Us



Benoît FORGETTE

Software and hardware Security Researcher
topic (Hardware/Android)



Sami BABIGEON

Software and hardware Security Researcher
topic (Hardware/CTF)

What is SightHouse?

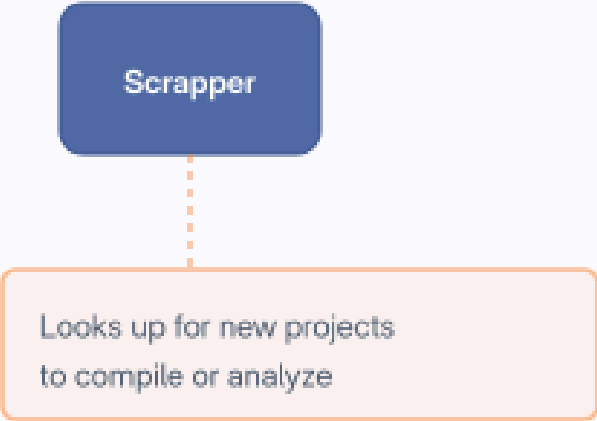
SightHouse is a tool designed to assist reverse engineers by **collecting** information and **metadata** from programs and recognizing **similar** functions.

Today's objectives:

- See how SightHouse can help you accelerate your Reverse Engineering workflow
- Learn how to create your own workers

SightHouse Workflow

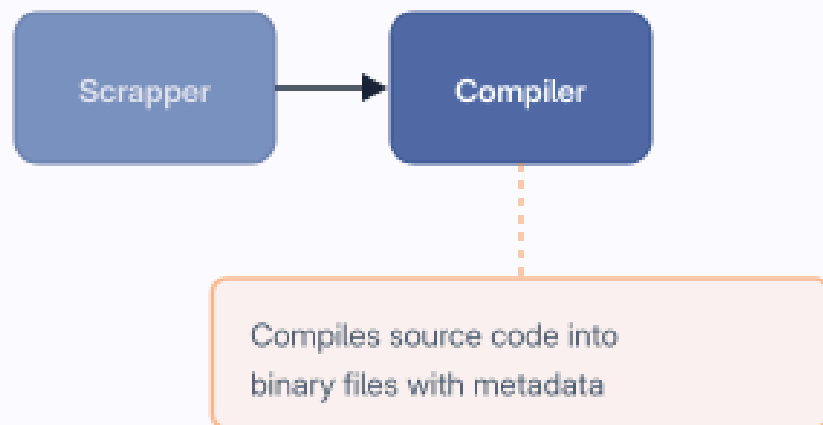
Scraper



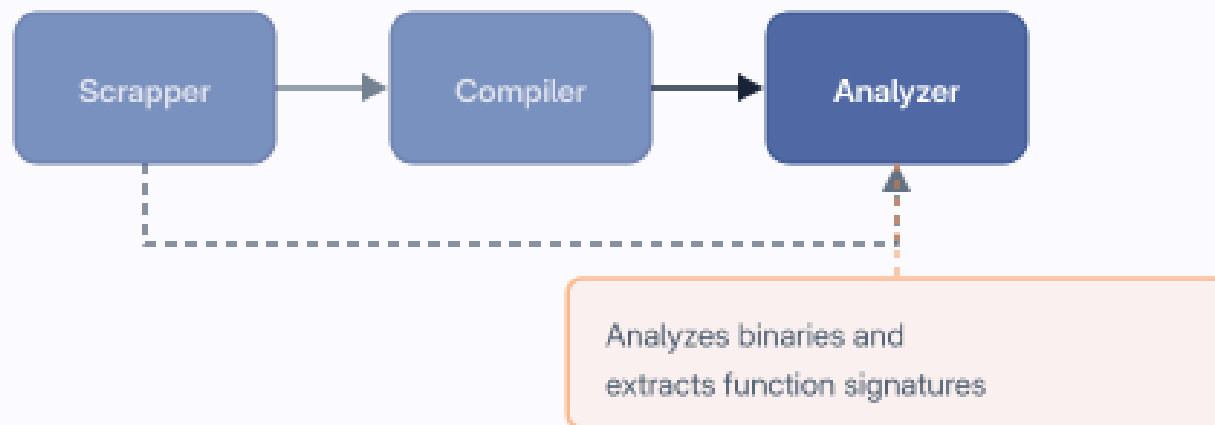
```
graph TD; A[Scraper] -.- B[Looks up for new projects to compile or analyze];
```

Looks up for new projects
to compile or analyze

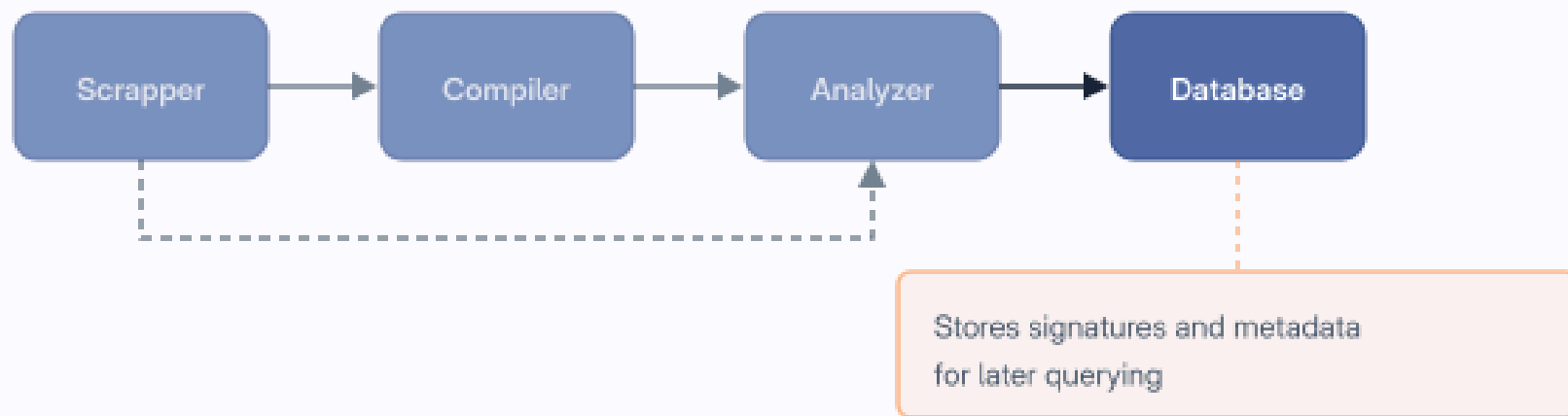
SightHouse Workflow



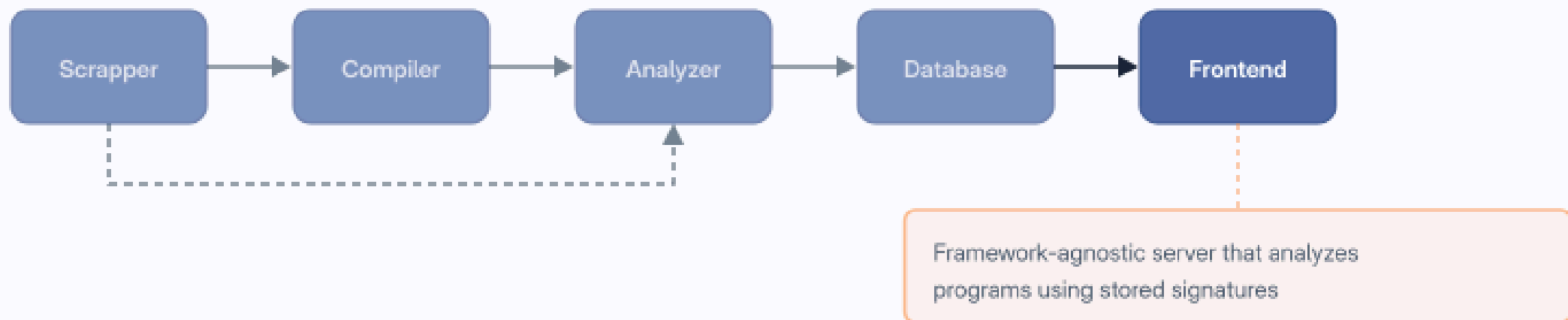
SightHouse Workflow



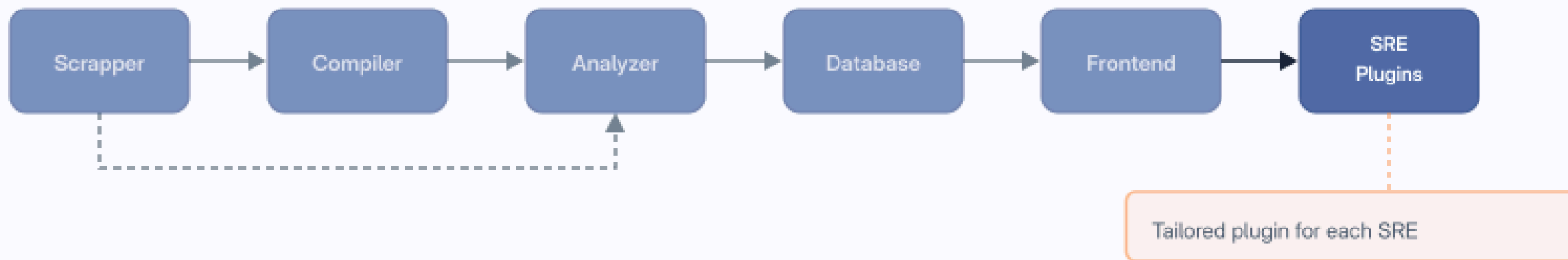
SightHouse Workflow



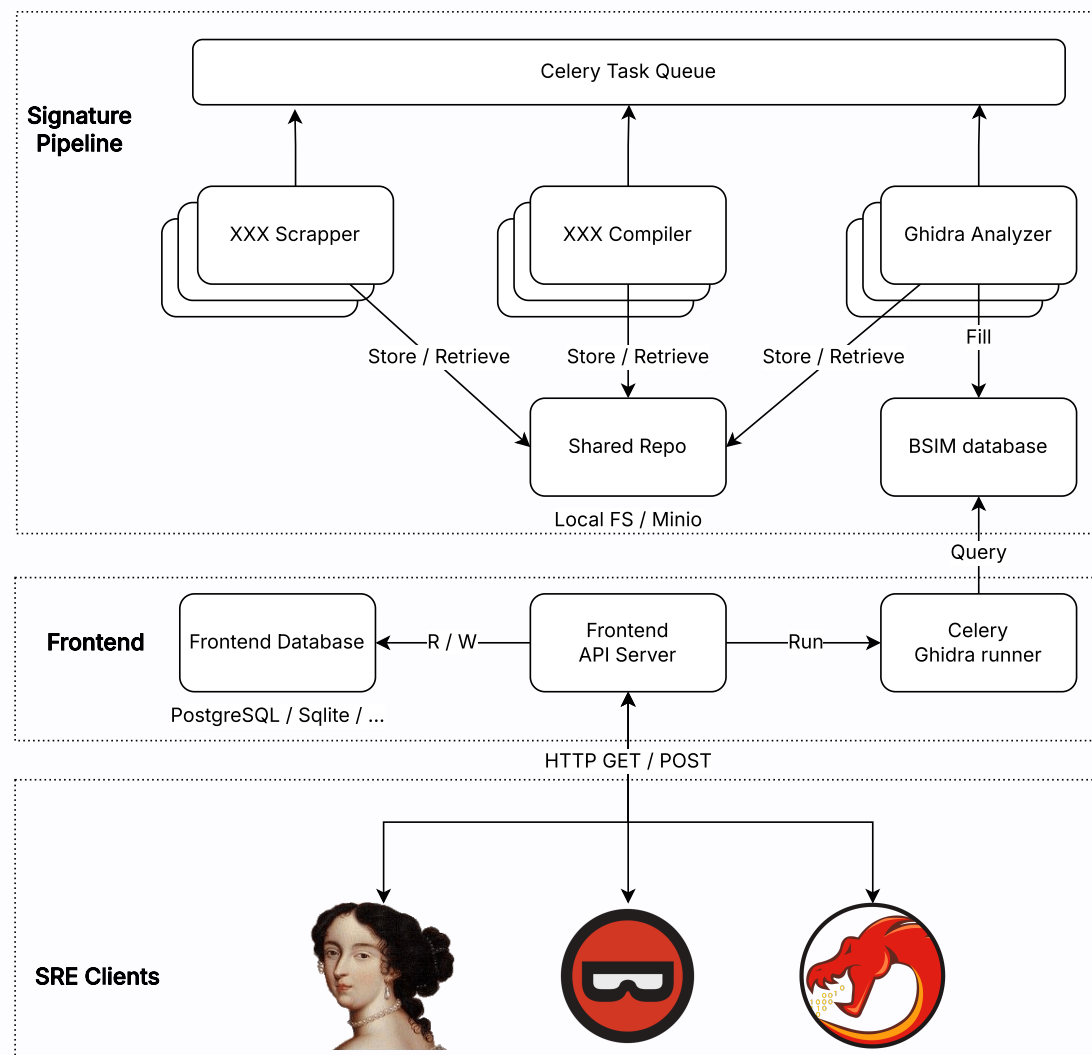
SightHouse Workflow



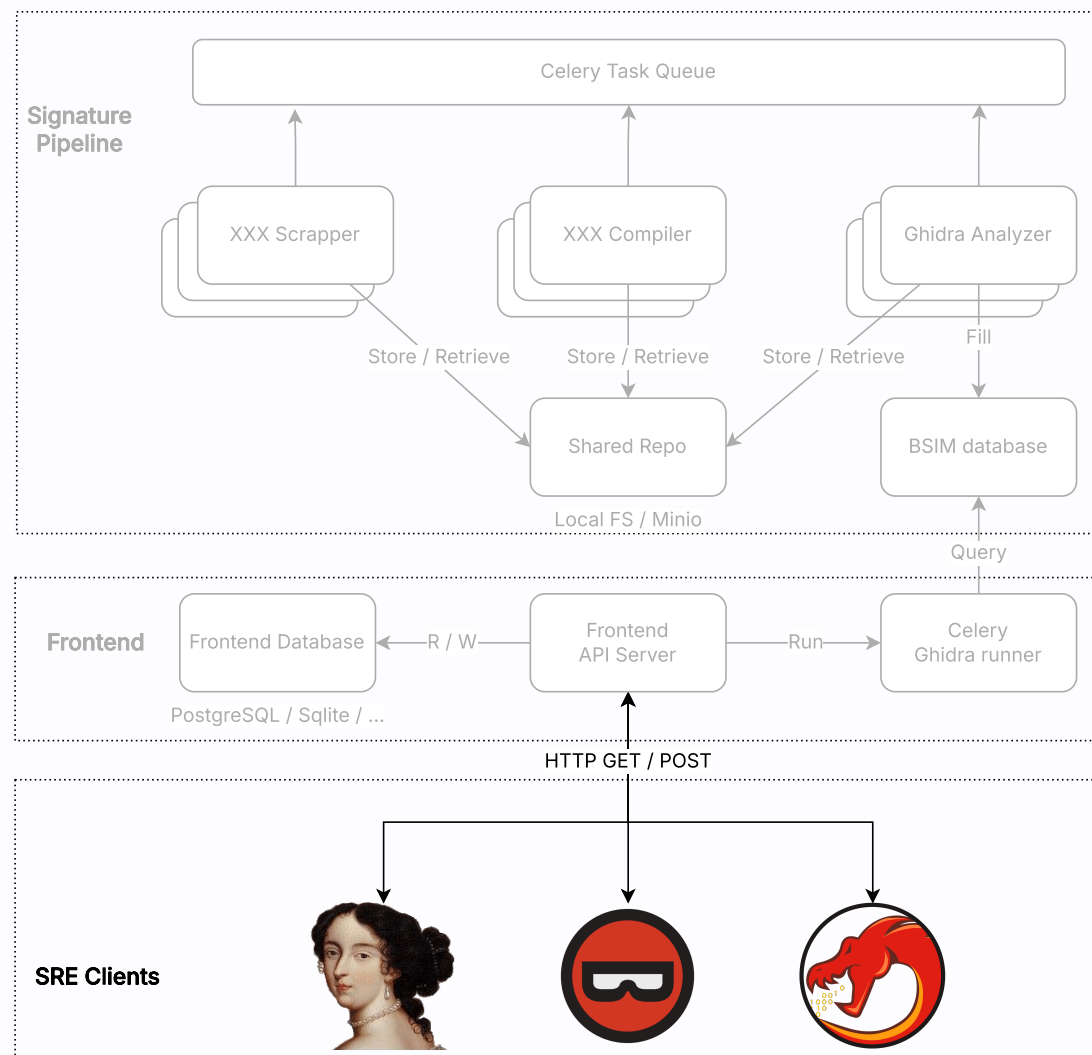
SightHouse Workflow



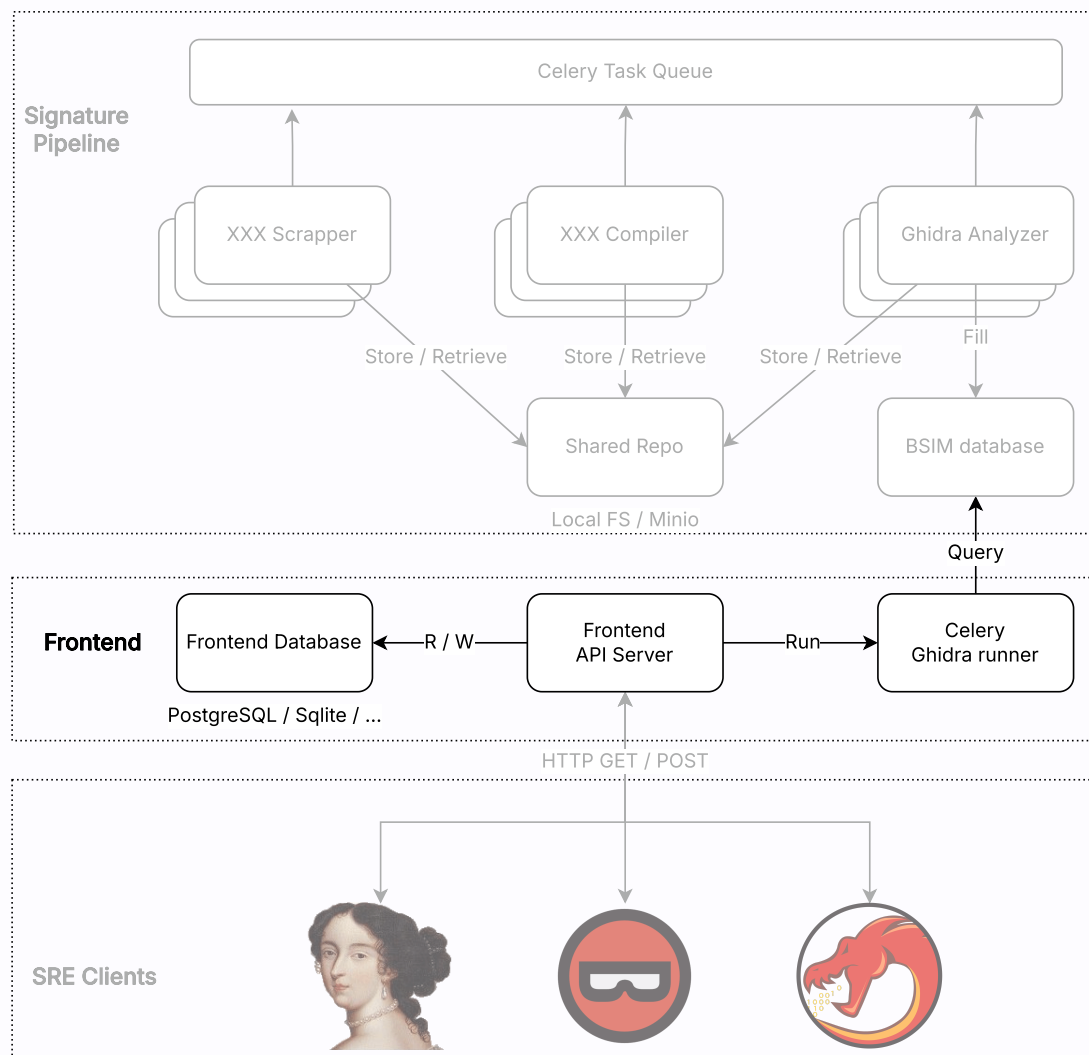
SightHouse Architecture



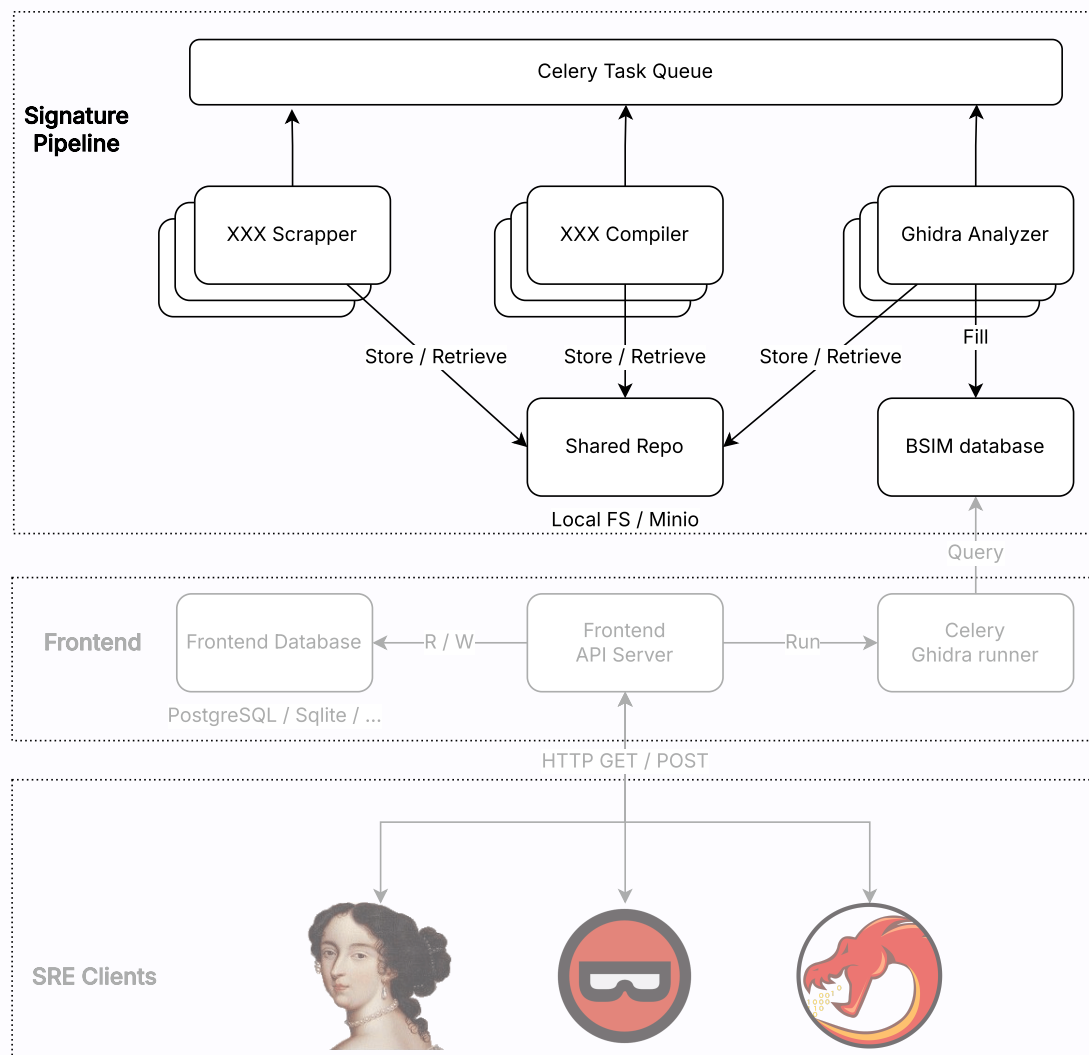
SightHouse Architecture



SightHouse Architecture



SightHouse Architecture





Quarkslab

Using SightHouse



Pass
the SALT

SRE Plugins

To interact with the **Frontend**, SighHouse offers plugins tailored to different SREs.



SRE Plugins

To interact with the **Frontend**, SighHouse offers plugins tailored to different SREs.



Installation

You will need **Ghidra 11.3** (or later) as it now comes with Python 3 support via pyGhidra.

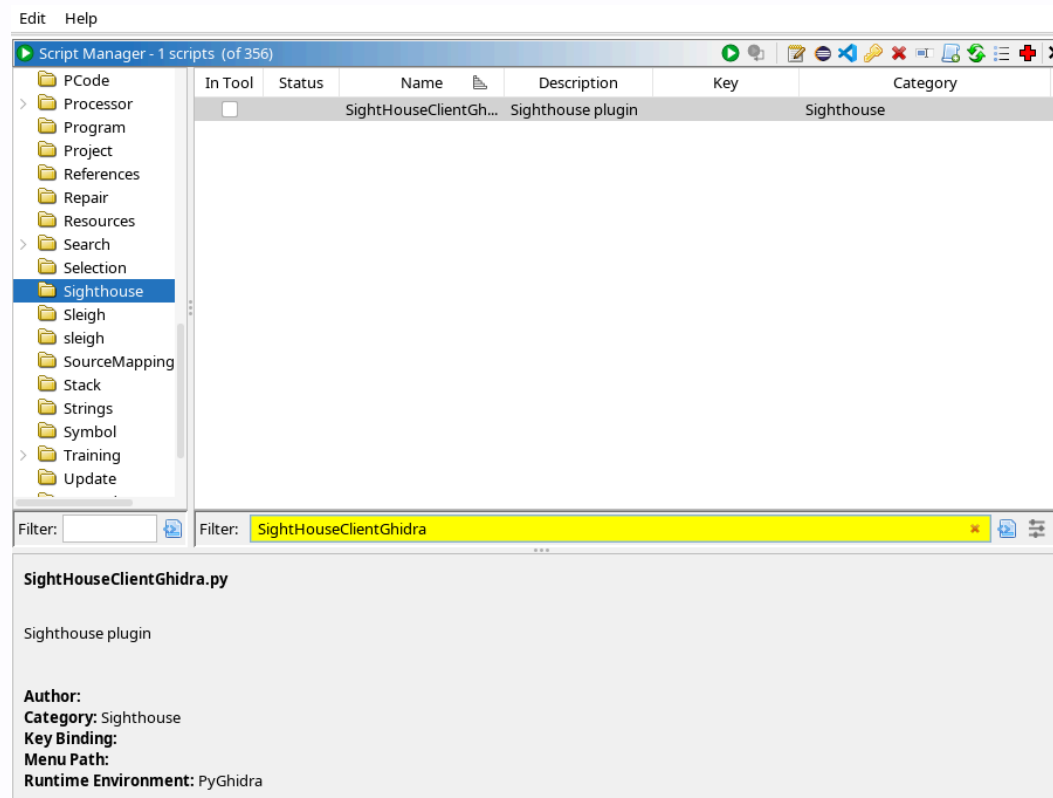
```
$ pip install sighthouse-client  
$ sighthouse client install ghidra --ghidra-install-dir /path/to/ghidra
```

Or for the other SRE:

```
$ pip install sighthouse-client  
# IDA  
$ sighthouse client install ida --ida-dir /path/to/ida_dir  
# Binja  
$ sighthouse client install binja
```

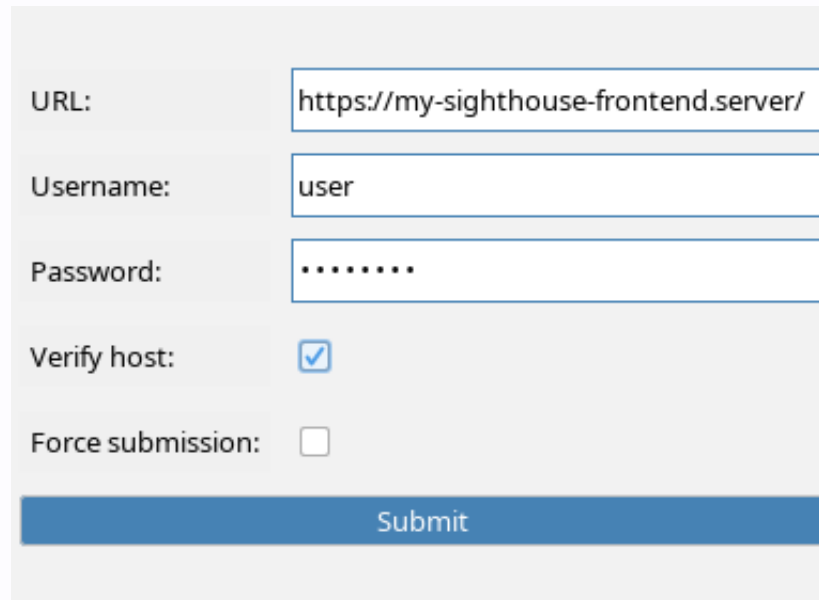
Running the Plugin

After restarting Ghidra, open a program and run `SightHouseClientGhidra.py` from the **Script Manager** window.



Running the Plugin

When running the script, you get prompted with the following window.



URL:

Username:

Password:

Verify host: ☒

Force submission: ☐

- URL: <https://frontend.sg/>
- Username & Password: Ask devs to create your account

Running the Plugin

Enter all the required information and press **Submit**.

```

/*
- Function: _ZN4mbed17LittleFileSystem28dir_seekEPv1
  SDK: ststm32@6.1.1
  https://dl.registry.platformio.org/download/platformio/platform/ststm32/6.1.1/ststm32-6.1.1.tar.gz
  Score: 10.3697
  Number of match: 4
- Function: _ZN4mbed16LittleFileSystem8dir_seekEPv1
  SDK: ststm32@6.1.1
  https://dl.registry.platformio.org/download/platformio/platform/ststm32/6.1.1/ststm32-6.1.1.tar.gz
  Score: 10.3697
  Number of match: 4
- Function: RiotCrypt_Sign
  SDK: ststm32@6.1.1
  https://dl.registry.platformio.org/download/platformio/platform/ststm32/6.1.1/ststm32-6.1.1.tar.gz
  Score: 10.3697
  Number of match: 2 */

void FUN_080adac8(byte *param_1,undefined8 *param_2,undefined4 param_3,undefined4 param_4)
{
    byte *pbStack_70;
    undefined4 local_6c;
    undefined1 *local_60;
    undefined4 local_58;
    undefined4 local_54;
    undefined1 auStack_50 [64];

    FUN_080bd766(&pbStack_70);
    local_54 = DAT_080adafc;
    local_58 = 0x40;
    local_60 = auStack_50;
    local_6c = 0x7fffffff;
    FUN_080074c8(&pbStack_70,param_1,param_2,param_4);
    return;
}

```

Lab 1: Reverse using SighHouse

1. Install the SighHouse plugin for your SRE:

<https://quarkslab.github.io/sighouse/clients/installation/>

2. Download the program: <https://workshop.sg/tp1-reverse.zip>
and load it in your favorite SRE:

- Base Address: **0x8010000**
- Language: **ARM 32 LE**

Objectives: Identify the different tasks of the firmware using SighHouse.

Hint

We need to find the system tasks:

- Embedded firmware -> Some kind of **real-time operating system** (RTOS)
- We are looking for a function with the following prototype:

```
int create_task(void *func_ptr, char *name, int stack, void *param, ...);
```



Quarkslab

Lab 1 Solution



Pass
the SALT

Solution

We need to find the system tasks:

- Embedded firmware -> Some kind of **real-time operating system** (RTOS)
- We are looking for a function with the following prototype:

```
int create_task(void *func_ptr, char *name, int stack, void *param, ...);
```


Solution

SightHouse identified some functions including `pxPortInitialiseStack`

```
//  
// - Function: pxPortInitialiseStack  
// SDK: STM32duino FreeRTOS@10.3.2  
// https://dl.registry.platformio.org/download/wisefang/library/WelldoneSTM32/1.0.1/WelldoneSTM32-1.0.1.tar.gz  
// Score: 17.1092  
// Number of match: 3  
  
int FUN_08015d7c(int param_1,uint param_2,undefined4 param_3)  
  
{  
    *(undefined4 *)(param_1 + -4) = 0x1000000;  
    *(uint *)(param_1 + -8) = param_2 & 0xfffffffffe;  
    *(undefined4 *)(param_1 + -0xc) = 0x8015da5;  
    *(undefined4 *)(param_1 + -0x20) = param_3;  
    *(undefined4 *)(param_1 + -0x24) = 0xfffffffffd;  
    return param_1 + -0x44;  
}
```

Solution

After a quick search, we can find the repo sources:

```
git clone https://github.com/FreeRTOS/FreeRTOS-Kernel
cd FreeRTOS-Kernel
grep -r 'pxPortInitialiseStack'
```

Solution

```

29  STATIC void prvInitialiseNewTask( TaskFunction_t pxTaskCode,
28      const char * const pcName,
27      const configSTACK_DEPTH_TYPE uxStackDepth,
26      void * const pvParameters,
25      UBaseType_t uxPriority,
24      TaskHandle_t * const pxCreatedTask,
23      TCB_t * pxNewTCB,
22      const MemoryRegion_t * const xRegions )
21  {
20      StackType_t * pxTopOfStack;
19      UBaseType_t x;
18
17      > #if ( portUSING_MPU_WRAPPERS == 1 )
16      #if ( portUSING_MPU_WRAPPERS == 1 )
15      {
14          /* If the port has capability to detect stack overflow,
13             * pass the stack end address to the stack initialization
12             * function as well. */
11          #if ( portHAS_STACK_OVERFLOW_CHECKING == 1 )
10          {
9              #if ( portSTACK_GROWTH < 0 )
8              {
7                  pxNewTCB->pxTopOfStack = pxPortInitialiseStack( pxTopOfStack, pxNewTCB
6              }
5              #else /* portSTACK_GROWTH */
4              {
3                  pxNewTCB->pxTopOfStack = pxPortInitialiseStack( pxTopOfStack, pxNewTCB
2              }
1              #endif /* portSTACK_GROWTH */
1986          }
1          #else /* portHAS_STACK_OVERFLOW_CHECKING */
2          {

```

Solution

```

STATIC TCB_t * prvCreateTask( TaskFunction_t pxTaskCode,
                             const char * const pcName,
                             const configSTACK_DEPTH_TYPE uxStackDepth,
                             void * const pvParameters,
                             UBaseType_t uxPriority,
                             TaskHandle_t * const pxCreatedTask )
{
    TCB_t * pxNewTCB;

    /* If the stack grows down then allocate the stack then the TCB so the stack
    if( pxNewTCB != NULL )
    {
        #if ( tskSTATIC_AND_DYNAMIC_ALLOCATION_POSSIBLE != 0 )
        {
            /* Tasks can be created statically or dynamically, so note this
            * task was created dynamically in case it is later deleted. */
            pxNewTCB->ucStaticallyAllocated = tskDYNAMICALLY_ALLOCATED_STACK_AND_TCB;
        }
        #endif /* tskSTATIC_AND_DYNAMIC_ALLOCATION_POSSIBLE */

        prvInitialiseNewTask( pxTaskCode, pcName, uxStackDepth, pvParameters, uxPriority, pxCreatedTask, pxNewTCB, NULL );
    }

    return pxNewTCB;
}

```

Solution

Now we can list the cross-references to `prvCreateTask`:

```
undefined8 FUN_0801bfb0(undefined4 param_1,undefined4 param_2,undefined4 param_3)
{
    int iVar1;

    iVar1 = 0;
    FUN_0801c3fa();
    if (*DAT_0801c48c != 0) {
        param_3 = 3;
        iVar1 = prvCreateTask(&LAB_0801c174+1,PTR_s_J6g_]j#_0801c494,0x618,0,3,DAT_0801c490);
    }
    if (iVar1 == 0) {
        prvYieldForTask(DAT_0801c498);
    }
    return CONCAT44(param_3,iVar1);
}
```



Quarkslab

Extending SightHouse



Pass
the SALT

Extending SighHouse

SighHouse is designed in a **modular** way, allowing you to **extend** and **reuse** components. Components are organized in **packages** (like Python ones).

Module structure

```
MyModule
├── __init__.py
└── package.yml
```

package.yml

```
name: MyModule
description: A cool module that I made for Pass The Salt
version: 1.0.0
author: Hackerman
```

Scraping

One can create their own scrapper by inheriting from the **Scrapper class**!

```
from sighthouse.pipeline.worker import Scrapper, Job

class MyScrapper(Scrapper):

    def do_work(self, job: Job) -> None:
        base_url = job.worker_args.get("base_url")
        for i in range(10):
            # Retrieve source, push into local storage
            file = self.do_scraping("{} / {}".format(base_url, i))
            job.job_data.update({
                "origin": base_url,
                "file": file,
                "version": "1.0.0",
            })
            self.send_task(job)
```


Compiling

Scrapped packages are then processed by **Compilers**.

```
from sighthouse.pipeline.worker import Compiler, Job

class MyCompiler(Compiler):

    def do_work(self, job: Job) -> None:
        variants = self.validate_compiler_variants(job.worker_args)
        for name, options in variants:
            with TemporaryDirectory() as tmpdir:
                # Download source and extract into temporary directory
                extract_tar(self.get_file(job.job_data.get("file")), tmpdir)
                # Build program (assuming makefile-based build)
                run_process(["make"], cwd=build_dir)
                # Push objects into local storage and send task
                build_files = list(tmpdir.rglob("**/*.o"))
                self.upload_objects_file(job, build_files, ...)
```

Analyzing

Binary files are then processed by **Analyzers** to extract signatures (**BSIM**).

```
from sighthouse.pipeline.worker import Analyzer, Job

class GhidraAnalyzer(Analyzer):

    def do_work(self, job: Job) -> None:
        args = self.parse_args(job.worker_args)
        self.log("Starting analysis of {}".format(job.job_data.get("hash")))
        with TemporaryDirectory() as tmpdir:
            # Download compiled files and extract into temporary directory
            extract_tar(self.get_file(job.job_data.get("file")), tmpdir)
            run_ghidra_script(
                self.ghidradir,
                "/path/to/SightHouseAnalyzerScript.java",
                tmpdir, ...
            )
```

`GhidraAnalyzer` extracts **BSIM** signatures — other analyzers can be added.

Lab 2: Create Your Own Compiler

1. Install the Sighthouse pipeline:

<https://quarkslab.github.io/sighthouse/signature-pipeline/installation/>

2. Download the template: <https://workshop.sg/tp2-pipeline.zip>

3. Run your worker like this:

```
$ sighthouse package run -f -i ./path/to/compiler \  
-w "redis://pipeline.sg:6379/" \  
-r "s3://admin:password@pipeline.sg:9000/frontend"
```

Objectives: Create your own compiler to integrate into the Sighthouse pipeline.



Quarkslab

Lab 2 Solution



Pass
the SALT

Solution

```
def do_work(self, job: Job) -> None:
    variants: List[Tuple[str, Dict[str, str]]] = self.validate_compiler_variants(
        job.worker_args
    )
    job_file = job.job_data.get("file")

    for name, options in variants:
        with TemporaryDirectory() as tmpdirname:
            tmpdir: Path = Path(tmpdirname)
            # download and extract the source from S3
            req = self.get_file(job_file)
            if not req:
                raise Exception("Could not download tar file: '{}'.format(job_file))
            if not extract_tar(req, tmpdir):
                raise Exception("Could not extract tar file")

            # compile every .c to a .o with gcc
            ...
```

Solution

```
...
# compile every .c to a .o with gcc
cc = options.get("cc") or "gcc"
cflags = options.get("cflags") or ""
self.log("Compiling variant '{}' with {}".format(name, options))
sources = list(tmpdir.rglob("**/*.c"))
if not sources:
    raise Exception("No .c source found to compile")
for src in sources:
    obj = src.with_suffix(".o")
    run_process(
        [cc, *cflags.split(), "-c", str(src), "-o", str(obj)],
        cwd=src.parent,
        timeout=self.TIMEOUT,
    )
build_files = list(tmpdir.rglob("**/*.o"))
if len(build_files) == 0:
    raise Exception("Build produce 0 file")
metadata = [
    [job.job_data["name"], job.job_data["version"] + "-" + name]
]
self.pack_and_send_task(job, build_files, metadata)
```

Solution

Run using:

```
$ sighouse package run -f -i ./path/to/compiler \  
-w "redis://pipeline.sg:6379/" \  
-r "s3://admin:password@pipeline.sg:9000/frontend"
```

Successfully installed package

```
[INFO/MainProcess] Connected to redis://192.168.0.198:6379//
```

```
[INFO/MainProcess] mingle: searching for neighbors
```

```
[INFO/MainProcess] mingle: sync with 1 nodes
```

```
[INFO/MainProcess] mingle: sync complete
```

```
[INFO/MainProcess] celery@hcof6I9hd5RXL2Wy ready.
```

```
[INFO/MainProcess] Task do_work[9894cbdd-52ab-4b3b-bf91-0964cfa15b28] received
```

```
[INFO/ForkPoolWorker-1] do_work[9894cbdd-52ab-4b3b-bf91-0964cfa15b28]: Compiling variant 'x86_64' with {'cc': 'gcc', 'cflags': '-O0'}
```

```
[INFO/ForkPoolWorker-1] do_work[9894cbdd-52ab-4b3b-bf91-0964cfa15b28]: Packing files
```

```
[INFO/ForkPoolWorker-1] do_work[9894cbdd-52ab-4b3b-bf91-0964cfa15b28]: Publish file: 9f844c53920aa0cd880.tar.gz
```



Quarkslab

Questions?

Documentation & GitHub

quarkslab.github.io/sighthouse
github.com/quarkslab/sighthouse



Pass
the SALT